

What is LiveTape.ai?

Production: <https://app.livetape.ai> (marketing site: <https://livetape.ai>)

The problem

Generating a trading idea is cheap — LLMs do that for free now. Running it correctly, continuously, against real markets is the hard part: live data ingestion at scale, deterministic time semantics under network jitter, lock-free concurrency between producer and consumer processes, multi-tenant scheduling, and a serialization format that's stable enough to store, version, and replay. That's the system built here — a side project I designed, built, and run solo, end to end, currently live in production processing real market data across crypto, equities, predictions and more.

The solution

LiveTape.ai is a real-time, multi-tenant strategy execution engine for financial markets: a Python backend evaluates a user-composed AST against live market data, at sub-50ms internal latency, on every tick — not just once per 1-minute bar close like most no-code trading tools.

Tick arrives → ring buffer → lock-free memory-mapped snapshot → AST resolution, all in-process with no DB on the hot path. End-to-end order latency also depends on brokerage round-trip time and any external signal-provider hop upstream of evaluation — both outside LiveTape's control.

Architecture, end to end

```
React (visual AST builder) → JSON AST → Postgres (BYTEA)
                                   ↓ loaded once per agent
FastAPI / asyncio → resolve(node, ctx) → condition results
                                   ↑
SnapshotMirror (seqlock, memmap)      Redis pub/sub → WebSocket →
React UI
                                   ↑
Ring buffer (NumPy SoA) ← SymbolWorker ← exchange WS feed
```

Frontend as compiler, not interpreter. The React app contains zero trading logic. Every condition a user builds in the visual editor (`'SMA(50)[1] > SMA(200)[1]'`) is constructed as a recursive JSON tree — an `'ExpressionNode'` AST — that the frontend only renders and edits. The backend's `'resolve(node, ctx)'` in `'strategy/resolver.py'` walks the same tree shape recursively, with both sides

validated against the same schema (Zod on the frontend, Pydantic on the backend) so there's one source of truth for the grammar.

Python without the usual performance tax. The hot path never relies on Python-level loops or locking. There's a standing project rule that no Python `for` loop may touch a full ring-buffer-sized array (172,800 elements) on the event loop — a loop that size blocks long enough to starve HTTP handlers into timeout — so every bulk operation on market data is vectorized NumPy (fancy indexing, `reduceat`, slice assignment), and cross-process coordination is a memory-mapped `seqlock` instead of an OS lock. Python is the orchestration language; the actual array math runs in C.

Serialization has three representations, not one. A strategy is authored as a `dict` over the API, persisted as UTF-8 JSON bytes in Postgres (`BYTEA`), and converted at agent-start time into a `np.uint8` array — the form the engine actually loads into memory. Three explicit, tested conversion functions (`rules_to_bytes` / `rules_from_bytes` / `rules_to_uint8`) sit on that boundary, so the on-disk format, the wire format, and the in-memory format can each evolve independently without a migration touching the others.

Lock-free IPC between the data-ingestion process and the evaluation process. Market data arrives via WebSocket and is bucketed into a fixed-size NumPy ring buffer (struct-of-arrays, $O(1)$ push, no shifting). That buffer is exposed to a separate consumer (the strategy-evaluation worker) through a memory-mapped file using a `seqlock` protocol: writers bump a sequence counter odd→even around each write, readers retry if they observe an odd or changing sequence. No OS-level locks, no shared-memory object — the file itself is the boundary, and it scales to many symbols without contention.

A deterministic logical clock, not `time.time()`. Every evaluation cycle runs against `ctx.current_time_s`, an engine-internal clock advanced monotonically from incoming market events (bar close > tick > a synthetic heartbeat injected only during detected data stalls). This is what makes backtests and live runs use identical code paths — in backtest, the same clock is simply set directly from replayed event timestamps instead of real-time arrival. Wall-clock time never leaks into resolver or executor logic.

A `bar_index` is a relative offset, not an array index. Indicators reference bars as `parent_offset + node.bar_index`, composed recursively as the AST nests — so

``SMA(50)[1]`` inside a larger expression resolves relative to wherever its parent landed, and the same subtree can be reused at different temporal offsets without rewriting it. This is the single invariant the whole resolver is built around.

Multi-tenant scheduling. A single asyncio event loop runs up to 50 independent agents (``AgentWorker``), each with its own ``AgentRuntime`` state and zero per-cycle DB access — state is loaded once at agent start and mutated in memory, with durable event logs flushed asynchronously off the hot path.

Stack

Python 3.13 / FastAPI / asyncpg / SQLAlchemy / Redis / NumPy on the backend;
TypeScript / React / Zustand / Zod / lightweight-charts on the frontend; Docker
Compose across dev/preview/prod, deployed on OCI ARM64 with Cloudflare
Workers fronting the static frontend.